



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: Grupo 7

No de Práctica(s): Practica 2

Integrante(s): 426058744

120002850

323064039

323071378

323214636

426059576

Semestre: 3er Semestre

Fecha de entrega: 4 de Septiembre de 2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	2
1.1. Objetivos	2
2. Marco Teórico	2
2.1. Estructuras de control iterativas	2
2.2. Estructuras condicionales	2
2.3. Validación de datos de entrada	3
2.4. Normalización de magnitudes y valor absoluto	3
3. Desarrollo	3
3.1. Control de aprobaciones.	3
3.2. Búsqueda del número mayor con restricción de variables	5
3.3. Análisis del uso y ejecución de <code>Math.abs()</code>	7
3.3.1. Funcionamiento técnico	7
3.3.2. Justificación algorítmica	7
3.4. Proyección tabular	8
4. Resultados	8
5. Conclusiones	10

1. Introducción

El procesamiento de flujos de datos discretos requiere mecanismos que garanticen la integridad en la captura, la delimitación estricta de variables en memoria y la generación estructurada de salidas numéricas. En esta práctica se resuelven tres problemas algorítmicos fundamentales: el conteo y validación de calificaciones, la búsqueda del valor numérico con mayor magnitud en una secuencia fija bajo la restricción estricta de utilizar únicamente tres variables de control y almacenamiento, y la producción de una tabla formateada que proyecta múltiplos decimales sucesivos a partir de una variable de control iterativa.

El dominio riguroso de las estructuras de control iterativas y condicionales constituye la base para el diseño de algoritmos eficientes en la programación estructurada y orientada a objetos. Es por eso que desarrollar código que intercepte entradas inválidas y que restrinja el consumo de variables auxiliares fomenta la optimización de recursos computacionales, previniendo comportamientos indefinidos y garantizando que el flujo de ejecución responda con precisión.

1.1. Objetivos

- Investigar con el LLM la implementación de un programa que resuelva la clasificación de calificaciones con validación interactiva bajo ciertas condiciones.
- Creación de un programa que implemente la determinación del valor absoluto máximo en una secuencia de enteros con memoria acotada.
- Desarrollar un programa que realice la proyección tabular de potencias de diez, verificando el comportamiento del flujo de control mediante pruebas de ejecución iterativas.

2. Marco Teórico

2.1. Estructuras de control iterativas

Las estructuras de control iterativas permiten ejecutar de manera repetitiva un bloque de instrucciones en función del valor de verdad de una expresión booleana [1]. Los ciclos con contador como `for` se emplean preferentemente cuando el número total de iteraciones se encuentra definido desde el inicio del proceso, mientras que estructuras como `while` y `do-while` condicionan la continuidad de la ejecución a evaluaciones dinámicas, garantizando esta última al menos una evaluación del ciclo antes de verificar la condición de permanencia.

2.2. Estructuras condicionales

La toma de decisiones en un algoritmo se gestiona a través de instrucciones de selección que evalúan condiciones lógicas y bifurcan el flujo hacia ramas de ejecución

mutuamente excluyentes [2]. El uso de condiciones como `if` o `if-else` permiten canalizar el tratamiento de variables según criterios numéricos específicos, posibilitando la actualización selectiva de contadores o la emisión de mensajes de estado al usuario.

2.3. Validación de datos de entrada

La validación de entradas consiste en verificar que los datos suministrados al programa se ajusten estrictamente a las restricciones del dominio del problema antes de permitir que alteren el estado de las variables internas [3]. La combinación de ciclos de repetición con comprobaciones booleanas establece un filtro donde cualquier valor fuera de rango es rechazado, solicitando de forma continua una nueva captura hasta asegurar la coherencia del dato ingresado.

2.4. Normalización de magnitudes y valor absoluto

El cálculo del valor absoluto permite obtener la magnitud escalar de un número real independientemente de su signo algebraico [2]. En algoritmos de comparación y búsqueda de valores extremos, la aplicación del valor absoluto estandariza el análisis de secuencias numéricas, permitiendo comparar la distancia de los números respecto al origen sin alterar la estructura fundamental del flujo de control.

3. Desarrollo

3.1. Control de aprobaciones.

La primera aplicación gestiona el registro de diez estudiantes evaluados bajo una escala binaria fija donde la calificación uno representa aprobación y la dos indica reprobación. El algoritmo inicializa dos acumuladores en cero y ejecuta un ciclo de diez iteraciones. En cada ciclo, una estructura de repetición interna captura la entrada del usuario y evalúa si coincide con las opciones válidas; de no ser así, reitera la solicitud de captura hasta obtener un valor admitido. Una vez validada la entrada, una estructura condicional incrementa el contador de aprobados o el de reprobados. Al finalizar el ciclo principal, se imprime el resumen general y, si el total de aprobados es igual o superior a nueve, se emite un mensaje indicando la bonificación al instructor. El flujo lógico correspondiente se ilustra en la Figura 3.1.

Para verificar el comportamiento de este módulo, se diseñó una prueba donde se ingresaron deliberadamente valores no válidos durante la captura de las primeras evaluaciones, seguidos de nueve aprobaciones y una reprobación, confirmando la activación del filtro de validación y la correcta emisión de la bonificación final.

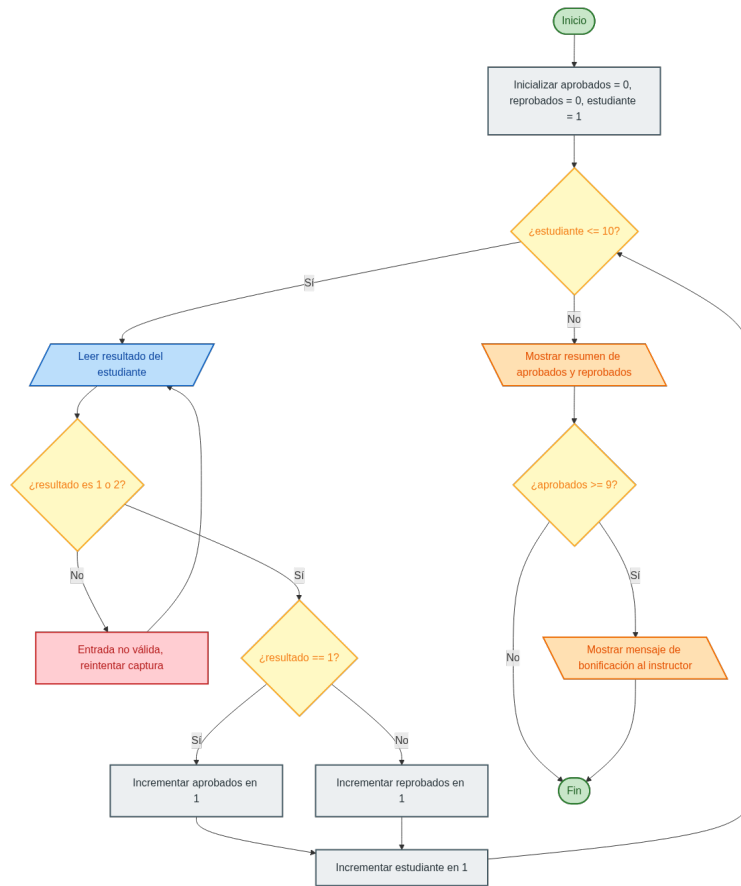


Figura 3.1: Diagrama de flujo para la determinación del entero mayor utilizando tres variables.Hola

3.2. Búsqueda del número mayor con restricción de variables

El segundo programa procesa una secuencia de diez números enteros introducidos por el usuario para identificar cuál de ellos posee la magnitud más alta, por diseño algorítmico, el sistema opera exclusivamente con tres variables: un contador de repeticiones, una variable para recibir la entrada actual y una variable para almacenar el mayor valor encontrado.

En cada una de las diez iteraciones del ciclo principal, se captura el número ingresado y se calcula de inmediato su valor absoluto, durante la primera iteración, dicha magnitud se asigna directamente a la variable de almacenamiento mayor; en las iteraciones consecutivas, se compara la entrada con el valor registrado y, si la supera, se actualiza la variable para que al término de las lecturas, se imprime el valor máximo alcanzado. La estructura del algoritmo se presenta en la Figura 3.2.

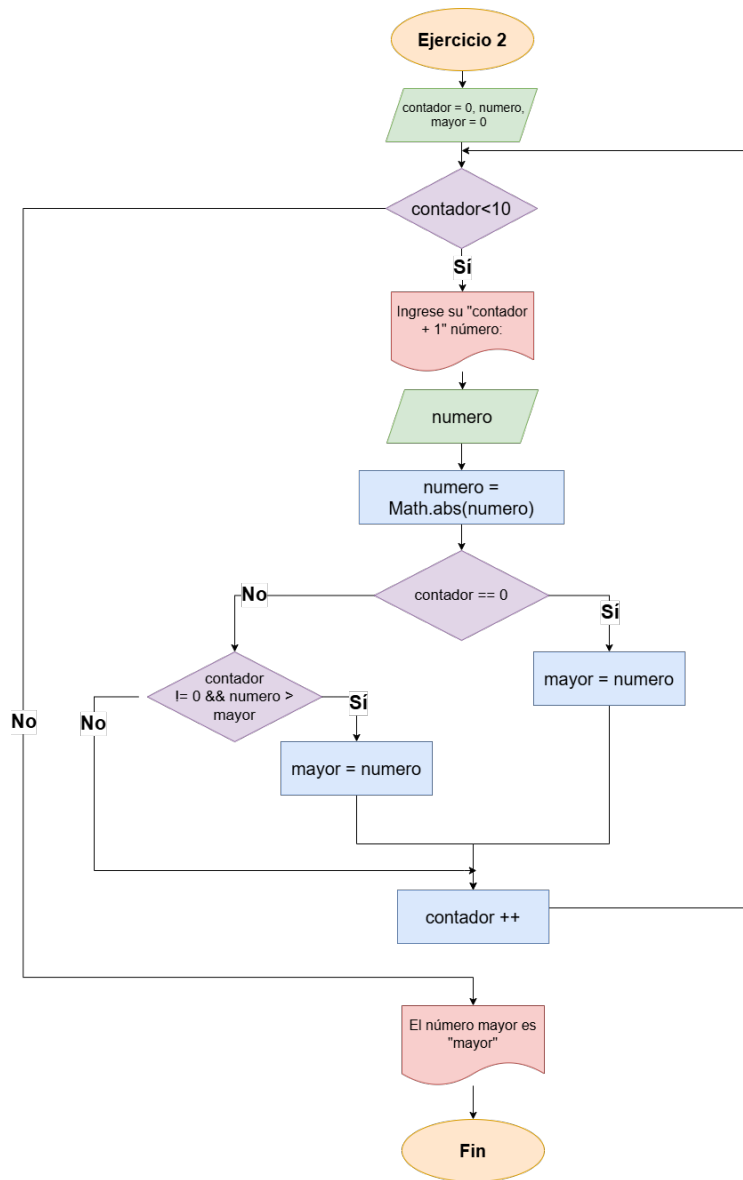


Figura 3.2: Diagrama de flujo para la determinación del entero mayor utilizando tres variables.

La prueba realizada consistió en introducir una lista mixta de diez números enteros positivos y negativos con magnitudes dispersas, comprobando que la función de valor absoluto normalizara los signos negativos y que la variable conservara con éxito la mayor magnitud registrada.

3.3. Análisis del uso y ejecución de `Math.abs()`

En el diseño del algoritmo para la búsqueda del número mayor (Ejercicio 2), se integró el método estático `Math.abs()` de la clase `java.lang.Math`. Esta función es crítica para el cumplimiento de los objetivos de normalización de datos y optimización de memoria.

3.3.1. Funcionamiento técnico

El método `Math.abs()` recibe como parámetro un valor numérico y devuelve su valor absoluto, definido matemáticamente como la magnitud del número sin tener en cuenta su signo, en la ejecución del programa, este proceso se realiza inmediatamente después de la captura del dato:

```
numero = leer.nextInt();  
numero = Math.abs(numero);
```

Esta secuencia transforma cualquier entrada negativa en su equivalente positivo (por ejemplo, -50 se convierte en 50), mientras que los valores positivos permanecen intactos.

3.3.2. Justificación algorítmica

El uso de esta función responde a dos necesidades fundamentales del problema planteado:

- **Criterio de Magnitud:** Al procesar una secuencia de números enteros, la aplicación del valor absoluto permite comparar la magnitud de los números, esto asegura que el programa identifique el valor con mayor valor escalar.
- **Optimización de Variables:** Dado que el problema restringe el uso a solo tres variables, `Math.abs()` permite pre-procesar la entrada en la misma variable de lectura (`numero`). Esto simplifica la lógica de la estructura condicional `if(numero > mayor)`, evitando comparaciones anidadas complejas para manejar casos negativos y positivos por separado.

En conclusión, la ejecución de `Math.abs()` actúa como un filtro de normalización en el flujo de datos, garantizando que la variable `mayor` almacene la magnitud máxima encontrada en las iteraciones.

3.4. Proyección tabular

El tercer ejercicio genera una tabla de múltiplos decimales para los primeros cinco enteros positivos, el algoritmo imprime inicialmente una línea de encabezados tabulados y posteriormente ejecuta un ciclo de conteo de cinco iteraciones.

En cada paso del ciclo, se toma el valor del índice incrementado en uno y se calculan simultáneamente sus productos por diez, cien y mil, dichos valores se envían a la salida estándar separados por tabulaciones para garantizar una alineación en columnas uniforme. El esquema de este proceso se detalla en la Figura 3.3.

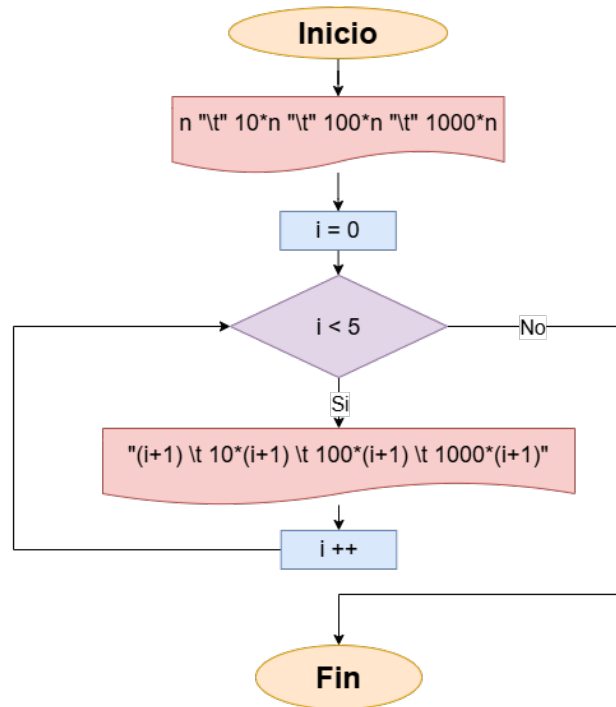


Figura 3.3: Diagrama de flujo de la generación tabular de progresiones numéricas.

La evaluación de este módulo consistió en la ejecución directa de la rutina en consola, verificando que la salida presentara exactamente cinco filas con las cuatro columnas debidamente alineadas y con los cálculos aritméticos correspondientes.

4. Resultados

Los programas implementados fueron ejecutados en un entorno de desarrollo para evaluar su desempeño ante los casos de prueba establecidos, en el primer ejercicio, el sistema solicitó sucesivamente las diez evaluaciones; tras ingresar entradas inválidas son rechazadas por el ciclo de validación, introduciéndose nueve calificaciones aprobatorias y una reprobatoria, produciéndose el resumen total y la leyenda de bonificación al docente, tal como se documenta en la Figura 4.1.

```

C:\Users\Personal\OneDrive\Desktop>javac AnalisisResultados.java

C:\Users\Personal\OneDrive\Desktop>java AnalisisResultados
Estudiante 1 (1=aprobó, 2=reprobó): 1
Estudiante 2 (1=aprobó, 2=reprobó): 1
Estudiante 3 (1=aprobó, 2=reprobó): 1
Estudiante 4 (1=aprobó, 2=reprobó): 1
Estudiante 5 (1=aprobó, 2=reprobó): 1
Estudiante 6 (1=aprobó, 2=reprobó): 1
Estudiante 7 (1=aprobó, 2=reprobó): 1
Estudiante 8 (1=aprobó, 2=reprobó): 1
Estudiante 9 (1=aprobó, 2=reprobó): 1
Estudiante 10 (1=aprobó, 2=reprobó): 2

--- Resumen ---
Aprobados: 9
Reprobados: 1
¡Se bonifica al instructor!

```

Figura 4.1: Captura de pantalla de la ejecución del control de aprobados, reprobados y bonificación.

Este es un ejercicio clásico de lógica de programación (comúnmente usado para enseñar estructuras de control como **while** o **for** y condicionales **if-else**).

Aquí te explico cómo funciona el programa paso a paso basándome en los 4 escenarios que te pidieron investigar:

Funcionamiento General del Programa

El programa utiliza un **ciclo** que se repite 10 veces. Dentro de cada repetición:

1. Pide un número al usuario.
2. Usa un **contador** (una variable que suma +1) solo si el número ingresado es un **1** (aprobó).
3. Al terminar las 10 repeticiones, el programa evalúa el valor final de ese contador con una **estructura condicional** (**if**).

Figura 4.2: Evidencia de respuesta del LLM.

En el segundo ejercicio, se introdujo una secuencia de diez valores enteros que incluyó números negativos, el programa procesó cada entrada a través del cálculo de valor absoluto y mantuvo actualizado el registro máximo empleando únicamente las tres variables permitidas, imprimiendo satisfactoriamente el valor mayor al término de la secuencia, como se observa en la Figura 4.3.

```
PS C:\Users\hecto\OneDrive\Desktop\alan_P00_git\Personal\AlanYescas> javac Ejercicio2_P2.java
PS C:\Users\hecto\OneDrive\Desktop\alan_P00_git\Personal\AlanYescas> java Ejercicio2_P2
Ingrese su 1 numero:
34
Ingrese su 2 numero:
234
Ingrese su 3 numero:
-55
Ingrese su 4 numero:
-600
Ingrese su 5 numero:
88
Ingrese su 6 numero:
543
Ingrese su 7 numero:
23
Ingrese su 8 numero:
70
Ingrese su 9 numero:
11
Ingrese su 10 numero:
42
El numero mayor es: 600
PS C:\Users\hecto\OneDrive\Desktop\alan_P00_git\Personal\AlanYescas>
```

Figura 4.3: Captura de pantalla de la determinación del número mayor en consola.

En el tercer ejercicio, la rutina iterativa produjo de manera inmediata la salida estructurada en columnas para los valores del uno al cinco junto con sus potencias decimales asociadas, verificándose la correcta alineación y precisión de los productos calculados, tal como se aprecia en la Figura 4.4.

```
C:\Users\Personal\OneDrive\Desktop\Yoyo_Repo>java Tabla
n      10*n      100*n      1000*n
1       10       100       1000
2       20       200       2000
3       30       300       3000
4       40       400       4000
5       50       500       5000
```

Figura 4.4: Captura de pantalla de la tabla de potencias decimales calculadas.

5. Conclusiones

El análisis de las pruebas de ejecución demuestra que la aplicación de estructuras iterativas y de condicionales permite resolver problemas de forma estructurada, la implementación del ciclo de validación en el control de calificaciones confirmó que la verificación de condiciones en las entradas previene los contadores acumulativos, garantizando la confiabilidad de las decisiones lógicas posteriores como el otorgamiento de incentivos docentes.

Por otra parte, la resolución de la búsqueda del valor extremo evidenció que es posible optimizar el consumo de recursos de memoria manteniendo la claridad algorítmica, la combinación del valor absoluto con una actualización condicional permitió procesar flujos de datos numéricos con signo sin requerir arreglos ni estructuras de almacenamiento complejas, cumpliendo estrictamente con la restricción de emplear un número estático de variables.

Finalmente, la generación tabular comprobó la eficacia de los ciclos con contador para proyectar matrices de datos, en conjunto, los tres programas cumplieron la totalidad de los objetivos planteados al articular los conceptos teóricos de control de flujo, validación interactiva y operaciones aritméticas dentro de soluciones computacionales eficientes.

Referencias

- [1] P. J. Deitel y H. M. Deitel, *Cómo programar en Java*, 10.^a ed. México: Pearson Educación, 2016.
- [2] L. J. Aguilar, *Fundamentos de programación: algoritmos, estructuras de datos y objetos*, 4.^a ed. Madrid: McGraw-Hill Interamericana, 2008.
- [3] L. J. Aguilar e I. Z. Martínez, *Programación en Java 6: algoritmos, estructuras de datos y objetos*. México: McGraw-Hill Interamericana, 2011.